GUÍA CONCEPTUAL · AGENTES DE IA

Memoria, estado y evaluación de agentes en GitHub

Cómo recordar, mantener el progreso, validar resultados y mejorar, usando Issues, Pull Requests, commits, Actions, logs y artifacts.

1 **Memoria**
Conserva la información relevante

2 **Estado**
Registra qué ocurrió y qué falta

3 **Evaluación**
Verifica criterios de éxito

4 **Mejora**
Ajusta instrucciones y herramientas

Instructor365.1

01 MEMORIA DEL AGENTE

Tres tipos de memoria

La memoria permite que un agente conserve la información necesaria para trabajar de manera consistente.



Corto plazo

Contexto de la tarea actual. Normalmente asociado a la sesión y no siempre se conserva entre sesiones.

- Instrucciones recientes
- Feedback recibido
- Decisiones inmediatas
- Pasos necesarios para continuar



Largo plazo

Conocimiento seleccionado que puede reutilizarse en tareas futuras.

- Decisiones importantes
- Patrones
- Conocimiento resumido
- Información reutilizable



Externa

Se guarda fuera del contexto interno del agente, en sistemas persistentes. En GitHub incluye:

- Issues
- Pull Requests
- Documentación
- Commits
- Logs
- Resultados de workflows
- Artifacts

02 DÓNDE ALMACENAR

Cada tipo de información, un lugar claro

Guardar cada dato en su ubicación evita ambigüedad y facilita retomar el trabajo.

Requisitos y criterios de aceptación	**Issue**
Decisiones y progreso	**Pull Request**
Reglas de validación	**Instrucciones del repositorio**
Resultados	**Workflow runs, logs y artifacts**
Procesos repetibles	**Instrucciones o skills reutilizables**

Guardar solo lo que afecta al resultado

- Requisitos
- Restricciones
- Decisiones
- Métodos de validación
- Criterios de aceptación

Evitar información innecesaria o duplicada reduce la confusión y el uso de contexto desactualizado.

03 ESTADO DEL AGENTE

Memoria recuerda; el estado registra

Memoria

Permite recordar qué información es importante.

Estado

Representa:

1. Qué se hizo
2. Qué decisiones se tomaron
3. Qué falta realizar
4. Qué resultados se obtuvieron

En GitHub, el estado se representa mediante:

Issues	Pull Requests	Commits	Workflow runs	Logs	Artifacts

Permiten conservar el progreso y continuar en otra sesión sin comenzar desde cero.

El Pull Request como punto central del estado

Un solo lugar para seguir el trabajo del agente y retomarlo en cualquier momento.

El PR puede contener

- Descripción de la tarea
- Criterios de aceptación o referencia al Issue
- Plan de implementación
- Decisiones tomadas
- Actualizaciones del progreso
- Commits
- Resultados de workflows
- Discusiones y revisiones

Al reanudar el trabajo, el agente puede:

1. Abrir el Pull Request existente
2. Revisar su descripción y el Issue relacionado
3. Verificar los commits realizados
4. Revisar los resultados de los workflows
5. Continuar desde el último estado conocido

05 CONTEXT DRIFT

Cuando el agente se desvía del objetivo

Context drift: las acciones del agente comienzan a desviarse del objetivo original o de decisiones tomadas anteriormente.

Cómo detectarlo

- ¿Los cambios cumplen los criterios de aceptación?
- ¿Los commits contradicen decisiones anteriores?
- ¿Existen workflows fallidos?
- ¿Faltan validaciones esperadas?

Cómo corregirlo

1. Revisar nuevamente el Issue o Pull Request
2. Comparar los cambios con los criterios de aceptación
3. Actualizar la descripción si las decisiones cambiaron
4. Ejecutar nuevamente las validaciones necesarias

Expiración, limpieza y reinicio

No toda la información debe conservarse indefinidamente.



Expiración

Los logs y artifacts de GitHub Actions tienen un período de retención y luego pueden eliminarse.

- Logs de GitHub Actions
- Artifacts



Pruning

La información antigua o innecesaria se elimina para evitar contexto obsoleto.

- Eliminar artifacts innecesarios
- Eliminar workflow runs obsoletos
- Evitar información redundante



Resumen

Un historial extenso se reduce a la información esencial.

- Actualizar el PR con las decisiones finales
- Referenciar commits importantes
- Vincular workflow runs en vez de duplicar logs



Reset

Se reinicia parte del contexto cuando:

- Cambian significativamente los requisitos
- Se selecciona un nuevo enfoque
- Las suposiciones anteriores dejan de ser válidas

07 CONTINUIDAD

Continuidad entre herramientas y entornos

Un agente puede empezar en un IDE, continuar en la CLI y terminar en GitHub.



Referencias persistentes para mantener la continuidad

- **Número del Pull Request**
- **Nombre de la rama**
- **SHA del commit**
- **URL del Issue**
- **URL del Pull Request**
- **Referencia a un workflow run**

GitHub como fuente de verdad compartida: cada entorno consulta el mismo estado persistente, en lugar de depender del historial temporal de una sesión.

Evitar contexto conflictivo y desactualizado

Contexto conflictivo

La misma información aparece en lugares distintos con valores diferentes. Para evitarlo:

- Definir una única fuente de verdad para cada información
- No duplicar requisitos innecesariamente
- Actualizar la fuente original cuando una decisión cambia
- Mantener sincronizados Issues y Pull Requests con el estado actual

Si cambian los criterios de aceptación, actualiza el Issue en lugar de definir criterios distintos en nuevos prompts.

Contexto desactualizado

Información que alguna vez fue correcta pero ya no refleja el estado actual. Sucede cuando:

- Una rama quedó desactualizada respecto de la base
- Un workflow corresponde a una versión anterior
- Cambiaron los requisitos
- Existen commits más recientes

Antes de continuar, revisar: commits recientes, estado actual de la rama, Issue y Pull Request, últimos workflow runs y resultados de validación.

09 EVALUACIÓN

Evaluar el trabajo con GitHub Actions

La evaluación determina si el trabajo está completo y es correcto; todo comienza con criterios claros.

Criterios de éxito claros

- La funcionalidad se comporta como se esperaba
- Los tests finalizan correctamente
- No se introducen nuevos problemas de seguridad

Se definen en el Issue y se referencian después desde el Pull Request.

Un workflow puede ejecutar

1	2	3	4	5
Tests	Linting	Compilación	Validaciones	Controles de seguridad

Los resultados aparecen asociados al Pull Request en

Status checks	Workflow runs	Pestaña Checks

Los workflows se almacenan normalmente en

```
.github/workflows/
```

10 QUALITY GATES

Impedir merges hasta cumplir las condiciones

Branch protection rules y otros mecanismos de protección del repositorio hacen cumplir estas políticas.



Condiciones de integración

- Tests obligatorios
- Status checks requeridos
- Revisión obligatoria
- Aprobación del Pull Request
- Rama actualizada antes del merge
- Restricciones sobre pushes directos

Flujo típico

1. Un Issue define la tarea y los criterios de aceptación
2. El agente crea una rama
3. El agente abre un Pull Request
4. GitHub Actions ejecuta los workflows
5. Los status checks muestran los resultados
6. Los checks obligatorios deben aprobarse
7. Un revisor aprueba los cambios
8. **El Pull Request puede integrarse**

11 SEGURIDAD

La seguridad como parte de la evaluación

GitHub ofrece mecanismos que pueden formar parte de los criterios de aceptación.



Code scanning

Analiza el código en busca de vulnerabilidades.



Dependency review

Revisa el impacto de las dependencias que cambian en un Pull Request.



Secret scanning

Detecta secretos expuestos en el repositorio.



Push protection

Bloquea los pushes que contienen secretos.

Los resultados de seguridad pueden ser requisito antes de permitir un merge.

Comparar la intención original con el resultado

Los agentes no siempre completan bien una tarea al primer intento. Estas son las fuentes para analizarlo:

Logs de Actions	Cambios del PR	Discusiones	Historial de commits	Resultados de workflows	Artifacts

Intención original

ISSUE

Requisitos y criterios de aceptación

PULL REQUEST

Plan y decisiones



Comparar

Resultado producido

- Commits
- Cambios de código
- Workflow outputs
- Logs

Tres categorías principales de fallo



Errores de razonamiento

El agente interpreta o decide incorrectamente.

EJEMPLOS

- Interpretar mal los requisitos
- Implementar una lógica incorrecta
- Ignorar criterios de aceptación



Uso incorrecto de herramientas

El agente usa mal los mecanismos disponibles.

EJEMPLOS

- Workflow mal configurado
- Comandos incorrectos
- Workflow que no se ejecuta cuando debería
- Permisos incorrectos



Problemas de contexto

El agente trabaja con información incorrecta, incompleta o desactualizada.

EJEMPLOS

- Utilizar un Pull Request desactualizado
- Perder decisiones anteriores
- Información contradictoria entre artifacts

Mejorar el comportamiento del agente

Una vez identificada la causa, se puede actuar en tres áreas.

Prompts e instrucciones

- Aclarar los criterios de aceptación
- Agregar restricciones
- Mejorar las instrucciones del repositorio

Memoria y estado

- Actualizar Issues y Pull Requests
- Eliminar información obsoleta
- Evitar contexto contradictorio
- Mantener una única fuente de verdad

Configuración de herramientas

- Corregir workflows
- Revisar eventos como push y pull_request
- Verificar permisos
- Configurar los checks necesarios

El ciclo de mejora continua

Un ciclo iterativo que mejora progresivamente la confiabilidad del sistema.

1. **Ejecutar el agente**
2. **Observar los resultados**
3. **Analizar logs y artifacts**
4. **Identificar la causa del fallo**
5. **Corregir prompts, memoria, estado o herramientas**
6. **Ejecutar nuevamente el workflow**
7. **Evaluar nuevamente el resultado**

Volver al paso 1: cada iteración parte de un sistema mejor ajustado

RESUMEN CONCEPTUAL

Memoria → Estado → Evaluación → Mejora

1

Memoria

Conserva la información relevante.

2

Estado

Registra qué ocurrió y qué falta hacer.

3

Evaluación

Determina si el resultado cumple los criterios establecidos.

4

Mejora

Analiza los fallos y ajusta instrucciones, memoria o herramientas.

Issues, Pull Requests, commits, GitHub Actions, logs y artifacts proporcionan la infraestructura para mantener estos elementos de forma persistente y trazable.

Un agente no debería depender solo de su contexto temporal: el repositorio y sus artifacts son una fuente de verdad persistente.